

Administrador Simplificado de Recursos: gestión de archivos y monitoreo del sistema en Windows y Linux

Fabrizio Espinoza Arce — Universidad Invenio

Licenciatura en Tecnologías de Información y Comunicación Empresarial

TIIT2007 – Sistemas Operativos | Septiembre de 2026

1. Resumen

Se construyó una herramienta de consola en C++17 que integra gestión básica de archivos y monitoreo de procesos y memoria del sistema anfitrión. Todo el código dependiente del sistema operativo está confinado en un único directorio, lo que permitió compilar y **ejecutar la misma herramienta en Windows y en Linux**: la suite de 26 pruebas pasa en ambos, con cero advertencias en MSVC /W4 y en g++ -Wall -Wextra -Wpedantic. Los tres casos de error del enunciado —archivo inexistente, permisos insuficientes y comando no disponible— se provocan de verdad y se prueban. Con 1000 archivos, crear, consultar metadatos y eliminar cuestan 0,84, 0,14 y 0,23 milisegundos por archivo, con un coste por archivo estable entre escalas. Listar 1000 archivos con todos sus metadatos tarda 1,5 milisegundos en total, casi cien veces menos que consultar esos mismos metadatos archivo por archivo: la diferencia entre una llamada al sistema por lote y una por archivo decide el rendimiento. Con la memoria del sistema al 86 % y los doce procesadores lógicos al 100 %, las operaciones se ralentizan entre 1,6 y 3,8 veces, 2,1 de media, sin ningún fallo. La herramienta ocupa 4,0 MB, el 0,025 % de la memoria del sistema, y sus lecturas coinciden con las de los comandos nativos de ambos sistemas.

2. Introducción

Un sistema operativo administra recursos —archivos, procesos, memoria— y expone esa información a través de interfaces muy distintas según el sistema. Una herramienta que quiera consultarlos debe elegir entre atarse a una plataforma o aislar las diferencias. Ese es el problema de ingeniería que este proyecto aborda.

Objetivo general. Implementar una herramienta de consola que integre gestión básica de archivos y monitoreo de procesos y memoria del sistema anfitrión.

Objetivos específicos. (a) Gestionar archivos —crear, listar, eliminar y mostrar metadatos— dentro de un directorio de trabajo controlado; (b) listar los procesos activos con su identificador, nombre y consumo; (c) reportar la memoria total, usada y disponible; (d) ofrecer un menú de consola que

navegue entre las tres funciones; (e) medir el rendimiento de las operaciones de archivo y su degradación bajo carga.

Alcance. La herramienta opera sobre un directorio de trabajo delimitado, no sobre todo el sistema de archivos. El monitoreo es una fotografía instantánea, no un seguimiento continuo. Se implementa para Windows y Linux, y ambas ramas se verifican.

Fuera de alcance. No se modifican permisos ni propietarios, no se terminan procesos, y no se monitorean recursos de red ni de disco.

3. Marco Teórico

3.1 Metadatos de archivo e inodos

Un sistema de archivos guarda, junto a los datos de cada archivo, una estructura con sus atributos. En los sistemas de tipo Unix esa estructura es el **inodo** y contiene el tipo, el tamaño, las marcas de tiempo, el propietario, el grupo y los bits de permiso [1]. Los tres metadatos que pide el enunciado —tamaño, fecha de modificación y permisos— son precisamente campos del inodo.

Windows no usa inodos: NTFS almacena los atributos en la Tabla Maestra de Archivos y el control de acceso mediante **listas de control de acceso (ACL)**, que asocian permisos a usuarios y grupos concretos con una granularidad que el modelo de nueve bits de Unix no puede representar [2].

Consultar los atributos tiene un coste que depende de **cómo** se pidan. Cada consulta por nombre es una llamada al sistema que debe resolver la ruta componente a componente y abrir el archivo [3]. Enumerar un directorio, en cambio, devuelve muchas entradas por llamada, y en NTFS cada entrada ya trae el tamaño, las fechas y los atributos [2]. Esa diferencia se mide en la sección 6.

3.2 Procesos y memoria

Un **proceso** es un programa en ejecución con su propio espacio de direcciones. El sistema operativo mantiene por cada uno una estructura de control con su identificador, su estado y sus recursos [1]. Tanto Windows como Linux exponen esa información mediante herramientas de línea de

comandos —`tasklist` y `ps`— además de sus interfaces de programación.

Sobre la memoria conviene distinguir dos magnitudes que a menudo se confunden. La memoria **libre** es la que no está asignada a nada; la memoria **disponible** es la que una aplicación podría pedir sin provocar intercambio a disco. Linux usa como caché la memoria que sobra, de modo que la libre suele ser muy baja aunque el sistema esté holgado [3]. Por eso este trabajo reporta `MemAvailable`, que es la estimación del propio núcleo y la que muestra `free -h` en su columna «available» [5].

Un último concepto explica los resultados bajo carga. Los procesadores con **multihilo simultáneo** presentan dos procesadores lógicos por núcleo físico; los dos comparten el núcleo, de modo que dos hilos ocupados en procesadores lógicos hermanos rinden bastante menos que dos núcleos completos [2].

3.3 Relación con la observación de xv6

La Guía de Análisis de la Unidad V dirige la lectura a `fs.c` y a la estructura `struct dinode` de xv6 [4]. Al compararla con lo que esta herramienta muestra aparece una ausencia significativa: el inodo de xv6 contiene el tipo, el tamaño y las direcciones de los bloques de datos, pero **no contiene propietario, grupo ni bits de permiso**. xv6 no tiene modelo de permisos en absoluto.

Esa observación explica algo que se midió en este proyecto. La herramienta muestra permisos en notación de nueve caracteres, y el resultado es distinto en cada plataforma: en Linux son los bits reales del inodo, en Windows son una aproximación sintetizada. La conclusión es la misma en los tres sistemas: **el modelo de permisos es una propiedad del sistema de archivos, no de la herramienta que lo consulta**. xv6 lo lleva al extremo al no tener ninguno.

4. Diseño de la Solución

4.1 Arquitectura

La herramienta se organiza en tres capas, como muestra la Figura 1.

```
main.cpp      orquesta; no consulta al SO
|
+-- src/io/    menu y presentacion
|             Menu, ReporteConsola, FormatoTabla
|
+-- src/plataforma/ TODO lo que toca el SO
|             Archivos.cpp      portable
|             SistemaWindows.cpp tasklist + API
|             SistemaLinux.cpp  ps + /proc/meminfo
|
+-- src/core/   logica pura: medicion,
               estadística, cronometro
```

Figura 1. Arquitectura en tres capas.

La decisión central es que **todo el código dependiente del sistema operativo vive en `src/plataforma/`**. La regla es

verificable: buscar la directiva `#include <windows.h>` fuera de ese directorio debe dar salida vacía, y esa salida vacía se conserva como evidencia en `results/pureza_plataforma.txt`.

Dentro de esa capa hay dos clases de fichero, y la distinción importa:

- **Archivos.cpp** tiene una sola implementación, escrita sobre `<filesystem>` de C++17. El estándar ya resuelve las diferencias.
- **SistemaWindows.cpp** y **SistemaLinux.cpp** son dos implementaciones de la misma interfaz. El estándar no cubre listar procesos ni consultar la memoria del anfitrión, así que no hay más remedio que duplicar.

Esa es exactamente la línea entre lo que hubo que duplicar y lo que no, y es la respuesta a qué habría que reescribir para portar la herramienta a un tercer sistema.

4.2 El directorio de trabajo controlado

El enunciado exige operar «dentro de un directorio de trabajo controlado». Eso no es solo una carpeta por omisión: es una frontera que hay que hacer cumplir. Sin comprobación, un nombre como `../../Windows/System32/algo` dejaría que la herramienta escribiera fuera de su área.

La comprobación normaliza la ruta y verifica que siga cayendo dentro de la raíz, **comparando componente a componente y no prefijos de cadena**. Comparar cadenas dejaría pasar un directorio hermano cuyo nombre empiece igual: `trabajo_otro` frente a `trabajo`. Además exige que el nombre designe algo **dentro** de la raíz y no la raíz misma: sin esa segunda regla, eliminar `.` borraría el propio directorio de trabajo cuando estaba vacío (defecto DEF-006, sección 6.6).

4.3 Manejo de errores

Toda operación de la capa de plataforma lanza una excepción con un mensaje que dice qué se intentaba y por qué falló, y el menú convierte cualquier excepción en un aviso y vuelve a mostrarse. Los tres casos del enunciado se distinguen:

- **Archivo inexistente:** se comprueba antes de operar y se nombra el archivo.
- **Permisos insuficientes:** el código de error del sistema se compara con `std::errc::permission_denied`, que unifica el `ERROR_ACCESS_DENIED` de Windows y el `EACCES` de Linux, y el mensaje lo dice con esas palabras.
- **Comando no disponible:** si `tasklist` o `ps` no se pueden ejecutar o terminan con error, se informa del comando exacto que se intentó.

4.4 Elección de mecanismo para los procesos

El enunciado permite invocar comandos del sistema o usar sus interfaces de programación. Se eligió lo primero, con `tasklist /FO CSV /NH` en Windows y `ps -e -o pid=,comm=,rss=` en Linux. Ambos comandos se piden en un formato sin encabezado y de columnas fijas, porque el formato por omisión **cambia con el idioma del sistema** y rompería el análisis de la salida en un Windows en español. Por la misma razón, la memoria de `tasklist` se lee descartando todo lo que no sea dígito: el separador de miles también depende del idioma. Los nombres de proceso pueden contener espacios, así que en Linux el nombre se toma como todo lo que queda entre el PID y la última columna.

5. Implementación

Lenguaje C++17, sin dependencias externas. Se compila con MSVC 19.51 en Windows y con g++ 15.2 en Linux, exigiendo cero advertencias en ambos.

Módulos. `GestorArchivos` implementa la gestión de archivos con la frontera de directorio; `Sistema.h` declara las consultas de procesos y memoria, con dos implementaciones; `Menu` conduce la interfaz de consola; `ReporteConsola` da formato a las vistas; `Cronometro`, `Medicion` y `Estadistica` sostienen las mediciones.

Código reutilizado. Conforme a las Consideraciones Transversales del enunciado se declara la reutilización de proyectos anteriores: el módulo de tablas ASCII, copia del Proyecto 1 al 3 cambiando solo el espacio de nombres, y las funciones de media y desviación muestral, extraídas del Proyecto 2 y usadas también en el 3.

Pruebas. Una suite propia de 26 pruebas, sin framework externo, que corre sin cambios en las dos plataformas. Para probar «comando no disponible» sin desinstalar nada, la capa de plataforma expone `listarProcesosCon()`, que acepta otro comando; la misma costura permite probar el análisis de salidas controladas. «Permisos insuficientes» se provoca retirando de verdad el permiso de escritura: con `icacls` en Windows y con los bits del directorio en Linux.

Compilación.

<code>build.bat</code>	compila (Windows)
<code>build.bat tests</code>	compila y ejecuta la suite
<code>make</code>	compila (Linux)
<code>make tests</code>	compila y ejecuta la suite

El menú recibe los flujos de entrada y salida por constructor en lugar de usar la consola directamente. Eso permite conducirlo desde un guion para generar evidencia sin que nadie teclee, y es lo que hace `EJECUTAR.bat`.

6. Resultados Experimentales

Entorno. Windows 11 Pro 10.0.26100, procesador AMD Ryzen 5 5600 de 6 núcleos y 12 procesadores lógicos, 15,9 GB de memoria, disco NTFS, MSVC 19.51 con `/O2`. La rama

de Linux se verificó en Ubuntu 26.04 LTS sobre WSL2, núcleo 6.6.87, con g++ 15.2 y `-O2`, en el mismo equipo.

Método. Archivos de 1 KB en cantidades de 10, 100 y 1000, que son las que fija el Dataset 4 del curso. Cada medición se repite cinco veces y se reporta media y desviación estándar muestral, medidas con `std::chrono::steady_clock`. Antes de medir se ejecuta una pasada de calentamiento que se descarta. Todo se regenera con `scripts/correr_experimentos.ps1`, y los datos crudos quedan en `results/csv/`.

6.1 Rendimiento de las operaciones de archivo

La Tabla 1 recoge los tiempos totales de cada operación en las tres escalas.

Tabla 1. Tiempo total por operación en Windows, media $\pm$ σ en milisegundos.

Operación	10 archivos	100 archivos	1000 archivos
crear	7,8 $\pm$ 0,3	77,9 $\pm$ 3,0	839,0 $\pm$ 147,7
listar	0,23 $\pm$ 0,30 †	0,36 $\pm$ 0,31 †	1,48 $\pm$ 0,32
metadatos	1,50 $\pm$ 0,04	14,2 $\pm$ 0,7	143,3 $\pm$ 5,9
eliminar	2,08 $\pm$ 0,07	24,2 $\pm$ 0,9	231,4 $\pm$ 14,1

† Desviación mayor que la mitad de la media: la herramienta marca estas mediciones como no fiables, y no se usan para sacar conclusiones.

La Tabla 2 divide cada total entre el número de archivos, que es lo que permite comparar escalas distintas.

Tabla 2. Coste por archivo en Windows, en milisegundos.

Operación	10	100	1000
crear	0,782	0,779	0,839
listar	0,023 †	0,004 †	0,0015
metadatos	0,150	0,142	0,143
eliminar	0,208	0,242	0,231

Crear, consultar metadatos y eliminar tienen un coste por archivo **estable**: entre las tres escalas varía como mucho un 16 %. Son operaciones lineales, en las que cada archivo cuesta al menos una llamada al sistema.

Listar se comporta distinto, pero hay que decirlo con cuidado. Con 10 y 100 archivos el tiempo es de décimas de milisegundo y la desviación lo iguala, así que esas dos cifras no sostienen ninguna comparación. La cifra fiable es la de 1000 archivos: **1,48 ms para enumerar 1000 entradas con su tamaño, fecha y permisos**. Consultar esos mismos metadatos archivo por archivo cuesta 143,3 ms, **97 veces más**, aunque la información obtenida es la misma. Es la diferencia descrita en la sección 3.1: la enumeración trae los atributos de muchas entradas por llamada, mientras que la consulta por nombre resuelve la ruta y abre cada archivo.

6.2 Comportamiento bajo carga

Se repitieron las mismas mediciones con el sistema cargado según los Laboratorios 4 y 6. El script aplica primero la carga de memoria —un proceso que reserva y escribe 4,1 GB—, espera a que confirme que terminó, y después lanza doce procesos de CPU en bucle, uno por procesador lógico. Solo empieza a medir cuando todos confirmaron que están en marcha. La Tabla 3 recoge el estado del sistema comprobado por el propio script.

Tabla 3. Carga del sistema durante el experimento.

Magnitud	Valor
Memoria disponible antes	6626 MB
Memoria ocupada por la carga	4386 MB
Ocupación de memoria al medir	86,4 %
Uso de CPU al empezar a medir	100 %
Uso de CPU al terminar	100 %
Procesos de carga vivos al terminar	13 de 13

La Tabla 4 compara cada operación consigo misma en las dos condiciones.

Tabla 4. Factor de ralentización bajo carga: tiempo con carga dividido entre tiempo sin carga.

Operación	10	100	1000
crear	1,84	1,82	2,20
listar	1,91 †	2,05 †	3,83
metadatos	2,11	1,96	2,37
eliminar	1,59	2,02	1,87

La ralentización va de 1,59 a 3,83 veces, con una media de 2,13; sin las dos filas no fiables, la media es 2,16. **Ninguna operación falló ni devolvió datos incorrectos:** solo tardaron más. El listado de 1000 archivos es la operación que más se degrada, lo que es coherente con que la presión de memoria reduce probablemente la caché del sistema de archivos, de la que el listado depende más que las demás, aunque con una sola escala fiable no se puede afirmar más.

6.3 Consumo de la propia herramienta

La Tabla 5 recoge lo que la herramienta gasta mientras opera.

Tabla 5. Recursos consumidos por el Administrador durante su operación.

Métrica	Valor
Memoria residente	4,0 MB
Sobre la memoria del sistema	0,025 %

Se mide con el mismo mecanismo que el resto de procesos, filtrando por el identificador propio: no hizo falta código adicional, y las cifras son directamente comparables con las

de cualquier otro proceso de la lista que muestra la opción 2 del menú.

6.4 Compatibilidad verificada en las dos plataformas

Tabla 6. Resultado de compilar y ejecutar en ambos sistemas.

Aspecto	Windows	Linux
Compilador	MSVC 19.51	g++ 15.2
Advertencias	0	0
Suite de pruebas	26/26	26/26
Crear, ms por archivo	0,779	0,051

La Tabla 6 muestra que el mismo código fuente compila y pasa la suite en los dos sistemas. Crear un archivo cuesta unas 15 veces más en Windows, con 100 archivos y el mismo método. No es una comparación limpia entre sistemas operativos —cambian el sistema de archivos, la capa de virtualización de WSL2 y el antivirus—, pero sí indica que el coste dominante no está en el código de la herramienta.

6.5 Casos de error provocados

Los tres casos del enunciado se provocaron sobre el binario real y quedan en results/errores_manejados.txt: un archivo inexistente, un directorio con la escritura denegada por icacls, y un comando inexistente. En los tres la herramienta muestra qué intentaba y por qué falló, y vuelve al menú.

6.6 Defecto detectado en la revisión

Una revisión posterior a la primera entrega encontró que eliminar ., con el directorio de trabajo vacío, **borraba el propio directorio**, porque la raíz superaba la comprobación de frontera. Se corrigió, y la prueba 2.6 lo impide ahora. La misma revisión encontró que la evidencia de carga de una versión anterior registraba solo 91 MB ocupados por la carga de memoria, y que la carga de CPU arrancaba después de empezar a medir. Ambos errores del script se corrigieron, y todas las cifras de esta sección provienen de la corrida corregida.

6.7 Validación frente a las herramientas del sistema

En el Laboratorio 6 (Semana 10) se construyó un panel de monitoreo con los comandos nativos de cada sistema. Ese panel sirvió para **validar** la herramienta: sus cifras se compararon con las de los comandos en el mismo instante. La Tabla 7 resume la comparación.

Tabla 7. Proyecto 4 frente a los comandos del sistema, en el mismo instante.

Magnitud	Proyecto 4	Sistema	Resultado
Linux, memoria usada	635,2 MB	free: 636,0 MB	Coincide

Magnitud	Proyecto 4	Sistema	Resultado
Linux, RSS de un proceso	107,5 MB	ps: 107,5 MB	Idéntico
Windows, memoria usada	61,8 %	CIM: 61,8 %	Idéntico
Windows, explorer	397,4 MB	WS 397,4 · privado 106,4	Igual al WS

Las memorias total y usada coinciden en los dos sistemas. El último renglón precisa **qué** mide la columna de memoria por proceso: *tasklist* reporta el *working set* completo, que incluye las páginas compartidas con otros procesos —las bibliotecas del sistema, por ejemplo—, y no solo las privadas. El RSS de *ps* en Linux tiene la misma propiedad. La consecuencia se recoge en las limitaciones de la sección 8.

7. Evaluación ISO/IEC 25010

Se aplican las cinco características de la rúbrica institucional, derivada de ISO/IEC 25010 [6].

Rendimiento (4). Se miden los tiempos de las cuatro operaciones en tres escalas, con media y desviación sobre cinco repeticiones y una pasada de calentamiento previa, y se comparan con y sin carga comprobada del sistema (Tablas 1 a 4). El reporte advierte automáticamente cuando la desviación supera la mitad de la media, y esas filas no se usan para concluir. El consumo propio es de 4,0 MB.

Fiabilidad (4). 26 pruebas automatizadas que pasan en las dos plataformas. Los tres casos de error del enunciado se provocan de verdad: archivo inexistente, permisos insuficientes retirando el permiso con la herramienta del sistema, y comando no disponible mediante una variante parametrizada de la consulta de procesos. El defecto de borrado de la raíz se detectó, se corrigió y quedó cubierto por una prueba. Bajo carga, ninguna operación falló. Las lecturas de memoria coinciden con las de los comandos del sistema (Tabla 7).

Usabilidad (4). Menú de consola con submenú para archivos, ayuda completa con ejemplos y códigos de salida documentados, confirmación antes de eliminar, y mensajes de error que dicen qué se intentaba y por qué falló. Una opción inválida no interrumpe el programa: informa y vuelve al menú.

Compatibilidad (4). La rúbrica admite compilar en múltiples entornos o declarar limitaciones con justificación. Aquí se cumple la primera rama: el mismo código compila y pasa la suite completa en Windows y en Linux, con la evidencia en `results/compatibilidad_linux.txt`. La limitación de los permisos en Windows se declara en la sección 8.

Mantenibilidad (4). Todo el código dependiente del sistema está confinado en un directorio, lo cual se verifica por

búsqueda automática. Añadir un tercer sistema operativo sería un archivo nuevo que implemente la misma interfaz.

8. Discusión de Resultados

Lo que decide el rendimiento es el número de llamadas al sistema. El resultado más útil de las Tablas 1 y 2 es la comparación entre listar y consultar metadatos: la misma información cuesta 97 veces menos obtenida por lote. Si la herramienta tuviera que manejar diez mil archivos, el cuello de botella serían las operaciones que hacen una o más llamadas por archivo —crear, consultar metadatos y eliminar—, y la optimización útil sería agruparlas o reutilizar la información de la enumeración, no acelerar el código propio.

La degradación bajo carga es la que predice la teoría. Con doce procesos ocupando los doce procesadores lógicos, la herramienta compite por un tiempo de CPU que ya está repartido. Si los doce procesadores fueran núcleos completos, el reparto justo daría una ralentización pequeña, de apenas 13 hilos sobre 12. Pero el Ryzen 5 5600 tiene **seis núcleos físicos** con multihilo simultáneo (sección 3.2): cuando los doce procesadores lógicos están ocupados, cada hilo dispone aproximadamente de medio núcleo. Un factor medio de 2,1 coincide con esa explicación. Lo relevante para una herramienta de monitoreo no es que no se degrade—eso sería imposible— sino que siga funcionando correctamente, y así fue.

Un experimento de carga necesita comprobar que la carga existe. La primera versión del script esperaba un tiempo fijo y suponía que la carga ya estaba aplicada. No lo estaba: la memoria ocupada fue una fracción de lo previsto y la CPU estaba casi ociosa al empezar a medir, pese a lo cual los resultados parecían plausibles. La versión corregida mide la carga antes y después de las mediciones, y deja un aviso en la evidencia si no se alcanza.

La portabilidad se consiguió aislando, no abstrayendo de más. Solo dos ficheros tienen implementación doble. La gestión de archivos, que a primera vista parecería lo más dependiente del sistema, resultó ser completamente portable gracias a `<filesystem>`.

Limitaciones. (a) En Windows, los permisos mostrados son una aproximación: la biblioteca estándar no traduce las listas de control de acceso y reporta `rw-rw-rw` salvo para archivos de solo lectura, mientras que en Linux se muestran los bits reales, `rw-r--r--` para un archivo nuevo. (b) En Windows, la biblioteca estándar elimina también los archivos marcados como de solo lectura. (c) La verificación de Linux se hizo sobre WSL2, que usa un núcleo Linux real pero no reproduce el hardware ni la carga de un servidor; por eso las cifras entre plataformas no son comparables en sentido estricto. (d) El monitoreo es una fotografía instantánea y no un seguimiento continuo. (e) La memoria por proceso incluye las páginas

compartidas (sección 6.7), así que la «memoria sumada de todos» que muestra la lista de procesos las cuenta varias veces y no equivale a la memoria usada del sistema, que se consulta en la opción 3 del menú.

9. Conclusiones

Se construyó una herramienta que cumple las cuatro funciones exigidas, que maneja los tres casos de error del enunciado y que funciona verificadamente en dos sistemas operativos, con la misma suite de pruebas pasando en ambos.

Tres resultados resumen el trabajo. Primero, el coste de las operaciones de archivo lo decide el número de llamadas al sistema: obtener los metadatos por lote es casi cien veces más barato que pedirlos uno a uno. Segundo, con la memoria al 86 % y la CPU al 100 %, la herramienta se ralentiza unas dos veces —lo que predice un procesador de seis núcleos con doce procesadores lógicos ocupados— y sigue siendo correcta. Tercero, la separación de la capa de plataforma redujo el código dependiente del sistema a dos ficheros, y eso es lo que hizo posible la verificación cruzada.

El aprendizaje metodológico más útil fue que **una medición bajo carga sin comprobar la carga no mide nada**, aunque sus números parezcan razonables. La misma revisión que lo detectó encontró un defecto de borrado que ninguna prueba cubría, lo que confirma el valor de probar los casos límite de la frontera y no solo los caminos esperados.

Extensiones futuras. Monitoreo continuo con muestreo periódico, lectura de las listas de control de acceso de Windows para reportar permisos reales, y reutilización de los metadatos de la enumeración para evitar consultas individuales.

10. Referencias

- [1] Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). John Wiley & Sons.
- [2] Tanenbaum, A. S. y Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson.
- [3] Arpaci-Dusseau, R. H. y Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces* (v1.00). Arpaci-Dusseau Books.
- [4] Cox, R., Kaashoek, M. F. y Morris, R. (2020). *xv6: a simple, Unix-like teaching operating system*. MIT.
- [5] Kerrisk, M. (2024). *proc_meminfo(5)* — *Linux manual page*. The Linux man-pages project.
- [6] ISO/IEC. (2011). *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.